SOFTWARE PARA EL ANÁLISIS DE DATOS: SPARK

Grado en Ciencia e Ingeniería de Datos

PSD

2024-2025

Oriol Parent (26066449C) y Lucas Viner (X6927982Z)

Introducción 3
Descripción del experimento 4
Entorno de ejecución 4
Datos de entrada 4
Aplicación analizada 5
Medición del rendimiento 5
Resultados 6
Resumen de resultados 6
Anàlisis detallado 7
Python Secuencial 7
Spark RDD 8
Spark DataFrame 8
Punto de Inflexión 9
Conclusiones de los resultados 10
Conclusiones 11
Bibliografía 12

Introducción

El presente proyecto tiene como finalidad estudiar y comparar el comportamiento de una aplicación analítica implementada en tres versiones distintas, con el objetivo de evaluar el impacto del modelo de programación y del entorno de ejecución sobre su rendimiento. Para ello, se han desarrollado tres variantes de la misma aplicación: una versión secuencial en Python, y dos versiones distribuidas sobre Apache Spark, una basada en la API de RDDs (coreSpark) y otra utilizando la API de DataFrames (SparkSQL).

La aplicación procesará un conjunto de datos derivados de la base pública IMDb, que recoge información sobre películas, actores, directores y sus calificaciones. Este fichero, con más de dos millones de registros, permite realizar un análisis de colaboraciones entre actores y directores, calculando tanto el número de películas compartidas como estadísticas agregadas sobre sus valoraciones.

Con ello, el proyecto persigue no solo identificar las diferencias de rendimiento entre las distintas implementaciones, sino también valorar la eficiencia de las interfaces de Spark disponibles y reflexionar sobre el uso adecuado de recursos computacionales en función del volumen de datos.

Descripción del experimento

El objetivo principal del experimento es evaluar el rendimiento de tres implementaciones de una misma aplicación de análisis de datos sobre un conjunto de registros derivados de la base de datos IMDb. Las tres versiones evaluadas son:

1) **Versión Python secuencial**: Implementación básica utilizando la biblioteca CSV de Python
2) **Versión Spark RDD**: Implementación utilizando la API de bajo nivel de Spark basada en RDDs (Resilient Distributed Datasets)
3) **Versión Spark DataFrame**: Implementación utilizando la API de alto nivel de Spark basada en DataFrames y SQL.

Entorno de ejecución

Para llevar a cabo la evaluación de rendimiento se ha desplegado un clúster de cuatro instancias virtuales en Amazon EC2, todas de tipo **m5.large**, cada una con 2 vCPUs y 8 GB de memoria RAM. El sistema operativo utilizado en todas las instancias es Ubuntu, y se ha ampliado el almacenamiento de cada nodo hasta los 30 GB. Las instancias se han configurado para permitir la comunicación entre ellas mediante un grupo de seguridad común con reglas de red apropiadas para el funcionamiento del clúster de Spark.

La instalación del software necesario en todas las instancias se ha realizado utilizando el script **automatic_deploy.sh**. Este script automatiza la instalación del software necesario y las claves SSH necesarias para permitir la autenticación entre nodos sin contraseña. Una de las instancias se ha designado como nodo master, mientras que las 3 restantes actúan como workers.

Datos de entrada

El conjunto de datos utilizado en el experimento es un fichero CSV denominado infoActors.csv, que contiene aproximadamente 2,3 millones de registros. Cada fila representa la aparición de un actor en una película, e incluye información sobre el identificador de la película (**tconst**), el identificador y nombre del actor, el identificador y nombre del director, la calificación media de la película (**averageRating**) y el número de votos (**numVotes**). A partir de este fichero se han generado versiones ampliadas (duplicadas, cuadruplicadas, etc.) mediante concatenación controlada para poder estudiar el comportamiento del sistema frente a distintos volúmenes de datos. Concretamente, se ha trabajado con los siguientes tamaños de ficheros:

- **Original**: Fichero base descargado de IMDB
- **x3**: Fichero original concatenado tres veces
- **x6**: Fichero original concatenado seis veces
- **x9**: Fichero original concatenado nueve veces

Aplicación analizada

La funcionalidad implementada en las tres versiones del programa consiste en identificar las colaboraciones entre actores y directores que han trabajado juntos en más de una película. Para cada pareja actor-director que cumpla esta condición, se calcula el número total de colaboraciones, la media de las calificaciones de las películas en las que han coincidido, así como la calificación mínima y máxima entre ellas. El resultado final muestra las 20 colaboraciones más frecuentes, ordenadas de mayor a menor número de películas compartidas.

La versión secuencial está escrita en Python utilizando el módulo **csv**, mientras que las dos versiones en Spark han sido implementadas por separado: una empleando transformaciones y acciones sobre **RDDs**, y la otra utilizando operaciones sobre DataFrames y consultas **SQL**. Todas las versiones han sido ejecutadas tanto en local (para pruebas iniciales) como en modo distribuido, utilizando el comando **spark-submit** con la URL del master del clúster. Todos los códigos usados se pueden visualizar en el zip de la entrega.

Medición del rendimiento

Para la evaluación de rendimiento, se han ejecutado todas las versiones sobre ficheros de entrada de distintos tamaños y se ha medido el tiempo total de ejecución utilizando el comando **/usr/bin/time**, recogiendo varias métricas pero poniendo el foco en el **elapsed time**.

Esta métrica refleja el tiempo total transcurrido desde el inicio hasta el final del proceso. Los resultados obtenidos se han registrado y representado gráficamente con el objetivo de analizar la escalabilidad y eficiencia de cada enfoque en función del tamaño del conjunto de datos.

Resultados

A continuación, se presentan y analizan los tiempos de ejecución obtenidos. El objetivo es evaluar el comportamiento de cada implementación según el tamaño de los datos de entrada y comprobar el punto en el que el procesamiento distribuido comienza a ofrecer ventajas sobre la versión secuencial.

Resumen de resultados

Tamaño	Versión	User (s)	System (s)	Elapsed (s)	%CPU
Original	**Python**	12.15	0.44	12.60	99%
Original	**RDD**	22.54	0.87	39.74	58%
Original	**DataFrame**	24.71	0.87	31.16	82%
x3	**Python**	37.80	0.55	38.36	99%
x3	**RDD**	23.70	0.91	66.50	37%
x3	**DataFrame**	24.95	0.82	36.15	71%
x6	**Python**	73.21	0.72	73.94	99%
x6	**RDD**	23.99	0.91	94.51	26%
x6	**DataFrame**	24.62	0.86	42.69	59%

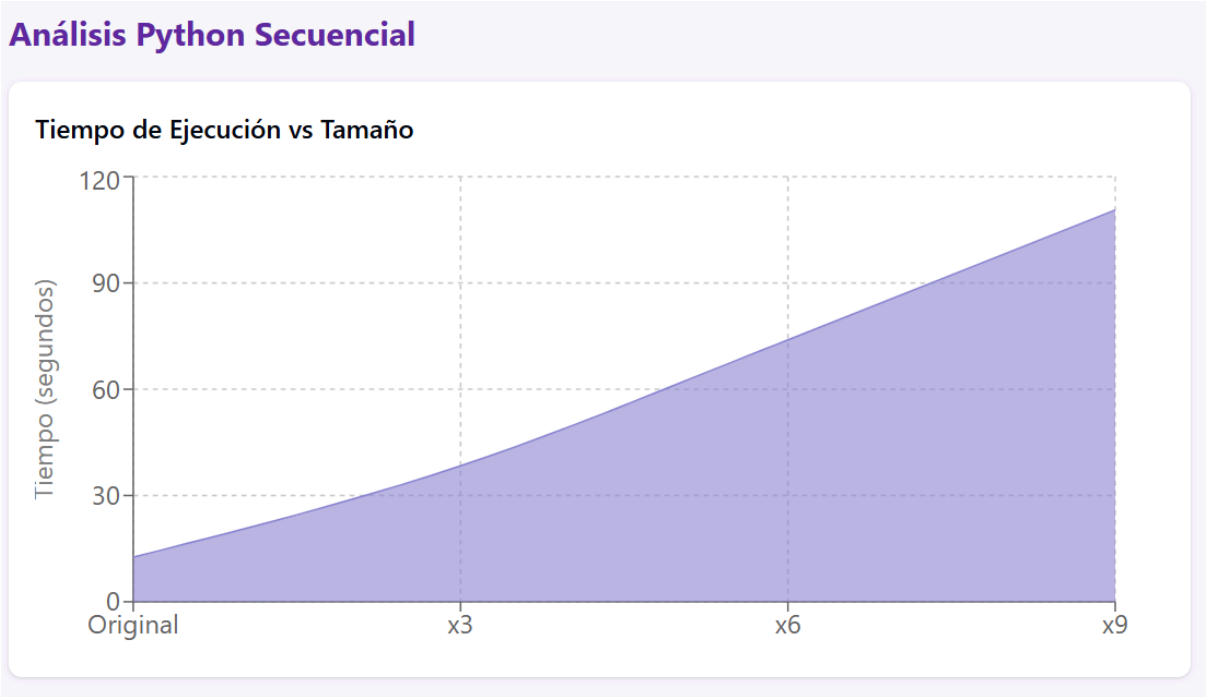
x9	**Python**	109.78	0.82	110.61	99%
x9	**RDD**	23.50	0.84	125.70	19%
x9	**DataFrame**	25.85	0.90	59.14	45%

Anàlisis detallado

Python Secuencial

La versión secuencial en Python muestra un crecimiento prácticamente lineal en el tiempo de ejecución conforme aumenta el tamaño de los datos. Es la opción más eficiente para el conjunto de datos original (el más pequeño) y mantiene un uso de CPU cercano al 100% en todos los casos, lo que indica un aprovechamiento óptimo de los recursos de un único núcleo. Además, el tiempo de usuario es prácticamente idéntico al tiempo total (elapsed), lo que sugiere una sobrecarga mínima de sistema.

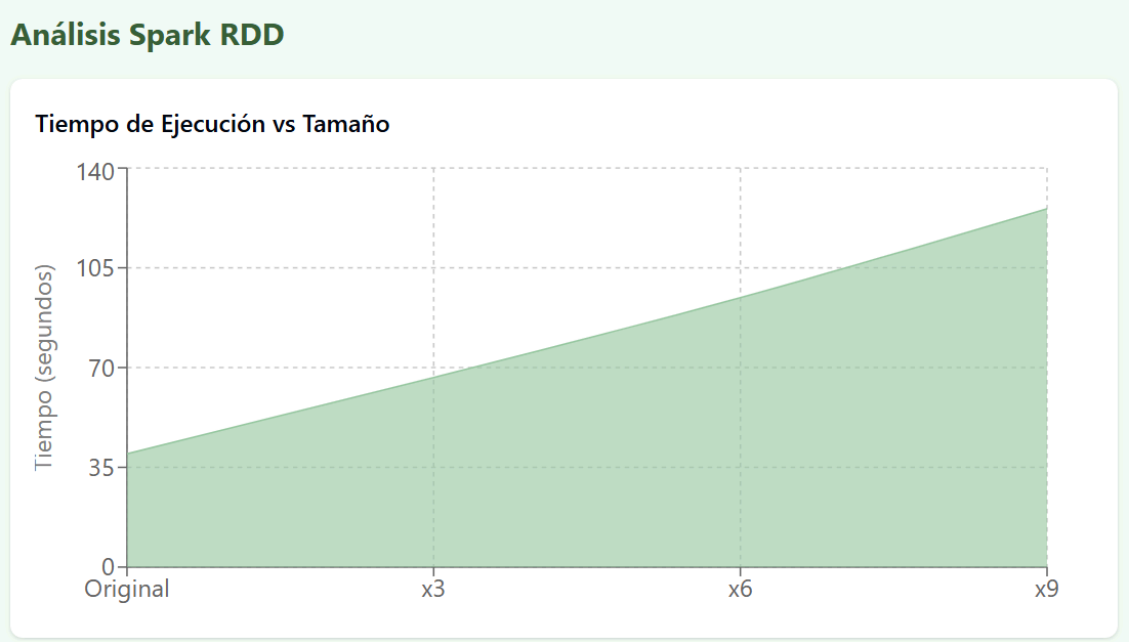
Sin embargo, a medida que el volumen de datos se incrementa, esta implementación se vuelve rápidamente ineficiente, con tiempos de ejecución que crecen proporcionalmente al tamaño de la entrada.



Spark RDD

La versión basada en RDD presenta una sobrecarga inicial significativa incluso para tamaños pequeños, debido a los costes de arranque del entorno distribuido y a la gestión manual de particiones y transformaciones. Aunque escala mejor que la versión secuencial para tamaños grandes, sigue mostrando incrementos notables en los tiempos de ejecución a medida que aumenta el volumen de datos.

Un aspecto destacable es que el porcentaje de uso de CPU disminuye drásticamente al aumentar el tamaño de los datos, pasando del 58% con el conjunto original a tan solo un 19% con el conjunto x9. Esto refleja que la mayor parte del tiempo en volúmenes grandes se destina a comunicaciones y sincronización entre nodos, no a cómputo efectivo. No obstante, el tiempo de CPU usuario se mantiene prácticamente constante, evidenciando que el tiempo de procesamiento real es estable y la penalización se debe a la infraestructura distribuida.

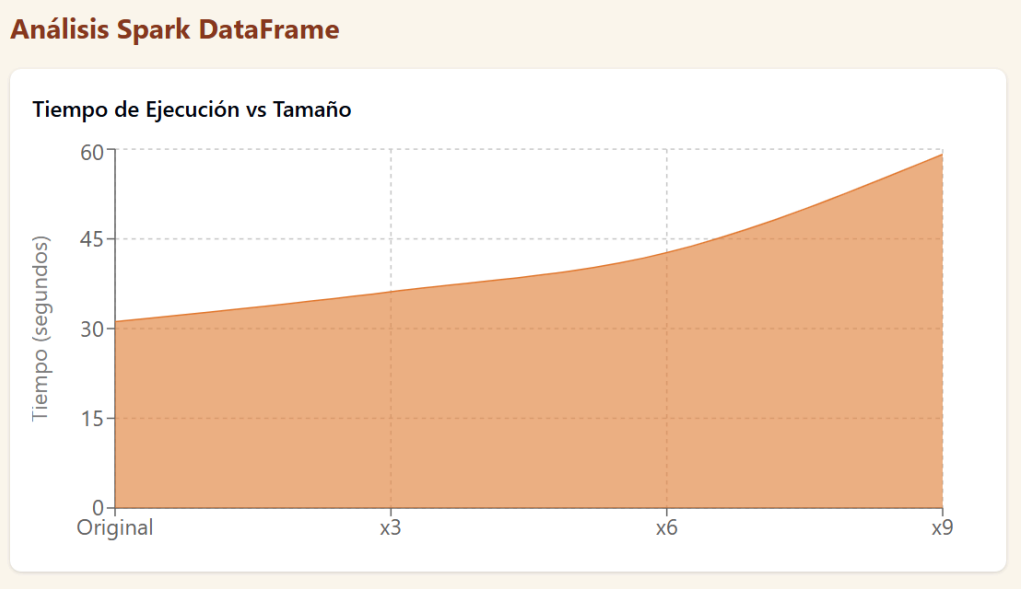


Spark DataFrame

La implementación con Spark DataFrames muestra el mejor comportamiento de escalabilidad entre las tres opciones. Aunque presenta una sobrecarga inicial similar a la de RDD, a partir del conjunto de datos x3 comienza a ofrecer ventajas significativas.

Es destacable que mantiene un tiempo de CPU usuario muy constante, en torno a 24-25 segundos, independientemente del tamaño de los datos.

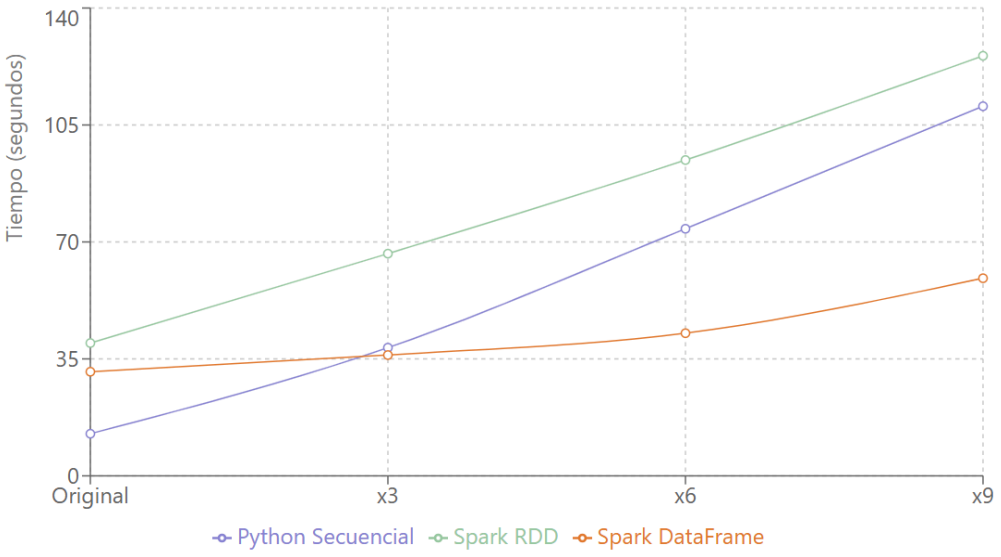
El porcentaje de uso de CPU disminuye al aumentar el volumen de datos, pero de forma mucho menos acusada que en la versión RDD, lo que se traduce en una mayor eficiencia global. Para los conjuntos más grandes (x6 y x9), DataFrames se consolida como la opción más rápida, superando tanto a Python secuencial como a RDD.



Punto de Inflexión

A partir de los resultados, se identifica un punto de inflexión claro en torno al conjunto de datos x3, donde el procesamiento distribuido comienza a superar al secuencial:

- Con el conjunto original, Python secuencial es la opción más rápida.
- Con el conjunto x3, Spark DataFrame prácticamente iguala el rendimiento de Python.
- A partir de x6, Spark DataFrame muestra una ventaja clara sobre las demás implementaciones.
- En el conjunto x9, DataFrames son aproximadamente un **47% más rápidos que Python secuencial** y un **53% más rápidos que RDD**, consolidando su superioridad en entornos de grandes volúmenes de datos.

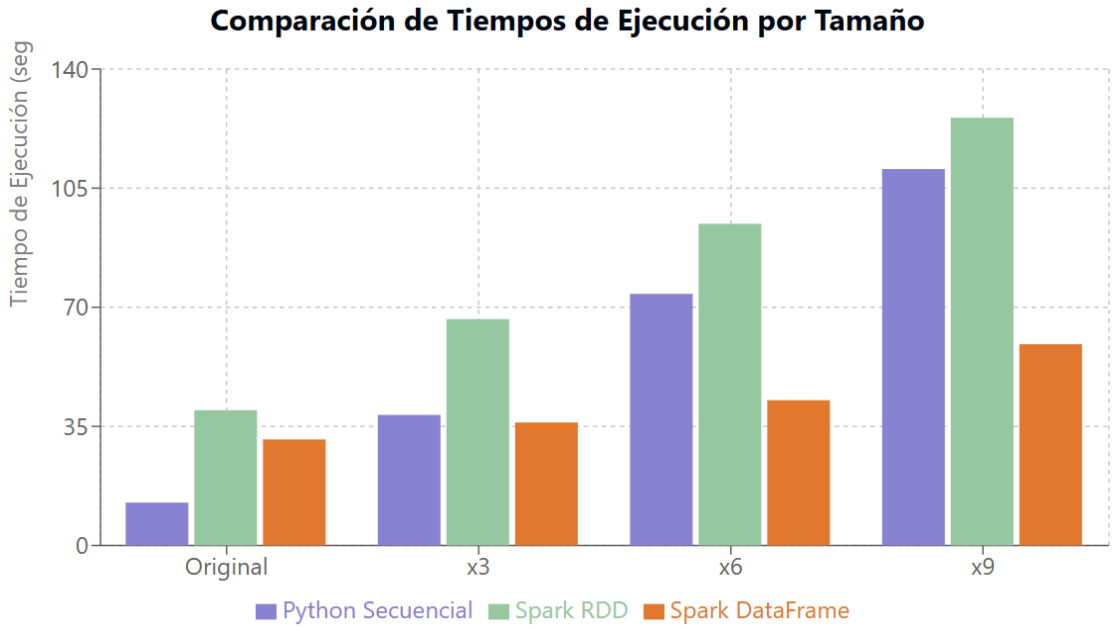


Conclusiones de los resultados

Los resultados de este experimento permiten extraer las siguientes conclusiones:

- **Sobrecarga inicial de Spark:** Tanto Spark RDD como DataFrame presentan una sobrecarga inicial apreciable, debida al tiempo necesario para iniciar y coordinar el entorno distribuido, lo que los hace menos eficientes para conjuntos de datos pequeños.
- **Ventaja de DataFrames sobre RDDs:** La API de DataFrame resulta notablemente más eficiente que la de RDD en todos los tamaños de datos, gracias a las optimizaciones que permite el motor Catalyst, ausentes en la API RDD.
- **Escalabilidad y tamaño de datos:** La implementación secuencial escala de forma estrictamente lineal, mientras que DataFrames muestran una escalabilidad mucho mejor, con incrementos de tiempo de ejecución proporcionalmente menores a medida que crece el tamaño de los datos.
- **Eficiencia en el uso de recursos:** Python secuencial utiliza un solo núcleo de manera intensiva (99% de CPU), mientras que las implementaciones distribuidas reparten la carga, reduciendo el porcentaje de uso de CPU por nodo pero aumentando la capacidad de procesamiento global.

En definitiva, este análisis confirma que el procesamiento distribuido no siempre es la opción óptima para cualquier volumen de datos, y que existe un punto de inflexión a partir del cual las ventajas de distribuir la carga superan a la sobrecarga que introduce la gestión distribuida.



Conclusiones

El proyecto ha permitido realizar un análisis comparativo entre diferentes enfoques de procesamiento de datos, evaluando su comportamiento en función del volumen de información tratado. A partir de los resultados obtenidos, se constata que la eficiencia de cada solución depende en gran medida del tamaño del conjunto de datos, mostrando la versión secuencial en Python un rendimiento claramente superior en entornos con volúmenes reducidos, gracias a su baja sobrecarga y elevado aprovechamiento de los recursos locales.

Sin embargo, al incrementar progresivamente la cantidad de datos, se observa cómo las implementaciones distribuidas basadas en Apache Spark comienzan a mostrar ventajas, especialmente a partir de un punto de inflexión situado aproximadamente en el tamaño x3 respecto al dataset original. En este contexto, Spark DataFrame ha demostrado ser la solución más eficiente, gracias a su menor tiempo de ejecución y mejor comportamiento en cuanto a escalabilidad.

En conclusión, este trabajo confirma que no existe una única solución óptima aplicable a todos los contextos, sino que es necesario valorar las características del problema, el volumen de datos y los requisitos de escalabilidad para seleccionar la tecnología y el enfoque más adecuados. Además, se ha evidenciado que el procesamiento distribuido no siempre resulta ventajoso, especialmente en entornos con cargas de trabajo reducidas, donde la sobrecarga asociada puede superar los beneficios obtenidos.

Bibliografía

Apache Software Foundation. (2024). *Apache Spark™ - Unified Analytics Engine for Big Data*. Disponible en: https://spark.apache.org/

Más allá de este, la bibliografía se basa exclusivamente en los apuntes y guiones de laboratorio proporcionados por el equipo docente de la asignatura durante el curso 2024–2025.